AI/ML Study Notes

Sample notes for testing the mini RAG bot. Topics: transformers, tokenization, prompting, embeddings, RAG, agents, core ML.

1. Transformers and Attention

The transformer architecture was introduced in the 2017 paper 'Attention Is All You Need' by Vaswani et al. It replaced RNNs for most language tasks because it trains in parallel and handles long-range dependencies better.

Self-attention: every token is turned into a Query, a Key and a Value vector using learned weight matrices. The attention score between two tokens is the dot product of one token's query with the other's key, scaled by sqrt(d_k) and passed through softmax to get weights that sum to 1. The output for a token is the weighted sum of all value vectors.

Intuition: the query asks 'what am I looking for?', the key advertises 'what do I contain?', and the value is the information that actually gets passed along.

Multi-head attention runs several attention heads in parallel. Each head can learn a different relationship (for example grammar, or which noun a pronoun refers to). Head outputs are concatenated and projected. The base model in the original paper used 6 encoder layers, 6 decoder layers, d_model of 512 and 8 heads.

Attention has no built-in sense of word order, so positional encodings are added to the token embeddings.

Causal masking: in GPT-style decoder-only models a token may only attend to earlier tokens, so the model cannot peek at the future while learning next-token prediction.

Three families: encoder-only (BERT, good for classification and embeddings), decoder-only (GPT, good for text generation), and encoder-decoder (T5, good for translation and summarization).

Main limitation: the cost of attention grows quadratically with sequence length, which is why long context windows are expensive.

2. Tokenization and How LLMs Work

LLMs do not read characters or words. Text is split into tokens, and each token is mapped to an integer ID, which is then looked up in an embedding table.

Byte Pair Encoding (BPE) builds the vocabulary by starting from bytes and repeatedly merging the most frequent adjacent pair into a new token. GPT-2 has a vocabulary of 50,257 tokens; GPT-4 style tokenizers (cl100k_base) have about 100,000.

Rule of thumb for English: 1 token is about 4 characters or 0.75 words. Non-English text, code and emojis usually need more tokens for the same content, which makes them more expensive and often harder for the model.

Many odd LLM failures trace back to tokenization: difficulty spelling words letter by letter, reversing strings, simple arithmetic, and weaker performance in languages that split into many tokens.

Vocabulary size trade-off: a bigger vocabulary gives shorter sequences but a larger embedding table and softmax layer, and rare tokens get little training signal.

The context window is the maximum number of tokens (prompt plus output) the model can handle at once.

Sampling controls: temperature 0 picks the most likely token (nearly deterministic), higher temperature adds randomness. Top-k keeps the k most likely tokens; top-p (nucleus sampling) keeps the smallest set whose probabilities add up to p.

Training stages: pretraining on huge text with next-token prediction, then supervised instruction tuning, then preference tuning such as RLHF. At inference the model generates one token at a time and uses a KV cache to avoid recomputing attention for earlier tokens.

3. Prompt Engineering

The DeepLearning.AI 'ChatGPT Prompt Engineering for Developers' course (Isa Fulford and Andrew Ng) is built on two principles: write clear and specific instructions, and give the model time to think.

Tactics for clear instructions: use delimiters (triple backticks, quotes or XML tags) to separate instructions from input text; ask for structured output such as JSON; have the model check conditions first; and give few-shot examples of the desired output.

Tactics for giving the model time to think: specify the steps to follow, and ask the model to work out its own solution before judging someone else's. Chain-of-thought prompting asks the model to reason step by step before the final answer.

Message roles: the system message sets behavior and rules, the user message holds the request, and the assistant message holds the model's replies. The model has no memory between calls, so the whole conversation is resent each turn.

Temperature: use 0 for reliable, repeatable tasks such as extraction and classification; use higher values for creative writing.

Prompt development is iterative: write a prompt, inspect the output, refine, repeat. There is no single perfect prompt.

Prompt injection happens when untrusted text (a web page, an email, a document) contains instructions that hijack the model. Delimiters and treating retrieved text as data help, but the problem is not fully solved.

Typical task patterns: summarizing, inferring (sentiment, topics, entities), transforming (translation, tone, format conversion) and expanding (drafting replies).

4. Embeddings and Vector Databases

An embedding is a dense vector of numbers that represents meaning. Texts with similar meaning get vectors that are close together, so searching for relevant text becomes finding the nearest vectors.

Dimensions: the sentence-transformers model all-MiniLM-L6-v2 produces 384-dimensional vectors and reads up to about 256 tokens; OpenAI text-embedding-3-small produces 1536 dimensions by default.

Similarity measures: cosine similarity (compares direction, ignores length), dot product, and Euclidean distance. Cosine distance equals 1 minus cosine similarity, so a lower distance means more similar.

The same embedding model must be used for both documents and queries. If you change the embedding model you must re-embed and re-index everything.

A vector database stores vectors together with metadata (for example page number or source) and supports fast similarity search. Options include FAISS (a Meta library, in-memory), Chroma (simple, local, persistent), Pinecone (hosted), Weaviate, Qdrant, and pgvector (a Postgres extension).

Exact nearest-neighbor search is slow at scale, so most systems use approximate nearest neighbor (ANN) indexes such as HNSW, trading a little recall for large speed gains.

Word2Vec and GloVe give static embeddings: one vector per word regardless of context. Transformer-based embeddings are contextual, so 'bank' in 'river bank' and 'bank account' gets different vectors.

TF-IDF is an older sparse keyword-weighting method that scores words by how frequent they are in a document and how rare they are across documents. BM25 is an improved ranking function and is still widely used, often combined with vectors in hybrid search.

5. Retrieval-Augmented Generation (RAG)

RAG means retrieving relevant documents first and then having the LLM generate an answer grounded in them. The term comes from a 2020 paper by Lewis et al. at Facebook AI Research.

Basic pipeline: load documents, split into chunks, embed the chunks, store them in a vector database, embed the user question, retrieve the top-k most similar chunks, put them in a prompt with the question, and generate the answer.

RAG versus fine-tuning: RAG is better for fresh or private knowledge, needs no retraining, and can cite sources. Fine-tuning is better for changing style, tone or output format, and for teaching consistent behavior.

Chunking trade-off: small chunks match a question precisely but give the LLM little context; large chunks give context but blur the embedding with unrelated content. A common starting guideline is 200 to 1000 tokens per chunk with 10 to 20 percent overlap.

Advanced retrieval techniques: sentence-window retrieval (match on single sentences, then send surrounding sentences to the LLM), auto-merging retrieval (small chunks inside larger parent chunks, merged when enough small ones match), hybrid search (keywords plus vectors), reranking with a cross-encoder, and query rewriting such as multi-query and RAG Fusion (combining result lists with reciprocal rank fusion).

Evaluation with the RAG Triad (used with TruLens): context relevance (are the retrieved chunks relevant to the question), groundedness (is the answer supported by the chunks), and answer relevance (does the answer address the question). An LLM is often used as the judge.

Common failure modes: the right chunk is not retrieved, the answer is split across chunks, the LLM ignores or misreads the context, the index is outdated, and PDF extraction produced garbled text.

Debugging habit: always inspect the retrieved chunks first to decide whether a bad answer is a retrieval problem or a generation problem.

6. Agents, Fine-tuning and Hallucinations

An AI agent is an LLM that decides what action to take next, uses tools such as web search, a calculator, code execution or APIs, observes the result, and repeats until the task is done.

The ReAct pattern (Yao et al., 2022) alternates Reasoning and Acting: the model writes a thought, chooses a tool call, reads the observation, and continues. Tool calling or function calling means the model outputs a structured request, the application runs it, and the result is returned to the model.

Multi-agent systems use several agents with different roles (for example planner, researcher, reviewer) that pass work to each other. Frameworks include LangChain, LangGraph and LlamaIndex.

Fine-tuning: LoRA freezes the original model weights and trains small low-rank adapter matrices, which cuts memory and cost a lot. QLoRA also quantizes the base model to 4 bits. These belong to a family called parameter-efficient fine-tuning (PEFT).

Quantization reduces numeric precision (for example 16-bit to 8-bit or 4-bit) to shrink memory use and speed up inference, with a small quality loss.

A hallucination is fluent text that is false or unsupported. Causes: the training objective is next-token prediction rather than truth, gaps or errors in the training data, and ambiguous prompts.

Mitigations: ground answers with RAG, require citations, instruct the model to say 'I don't know' when the context lacks the answer, use low temperature, and verify important outputs. RAG reduces hallucination but does not eliminate it.

MLOps basics: version your data, code and models, monitor model quality and drift after deployment, and automate training and deployment with CI/CD.

7. Core Machine Learning and Deep Learning

Gradient descent updates parameters in the direction that reduces the loss: new weight = old weight minus learning rate times gradient. A learning rate that is too large can diverge; too small makes training very slow. Backpropagation computes the gradients layer by layer using the chain rule.

Overfitting means the model fits training data very well but generalizes poorly. Remedies: more data, regularization (L1 or L2 penalty), dropout, early stopping, data augmentation, and a simpler model. Underfitting is the opposite problem and needs a more expressive model or longer training.

Bias-variance trade-off: high bias means underfitting, high variance means overfitting. Compare training error and validation error to tell which one you have.

Classification metrics: precision is TP divided by (TP + FP); recall is TP divided by (TP + FN); F1 is the harmonic mean of precision and recall. Accuracy is misleading on imbalanced data, so use precision-recall, F1 or ROC-AUC, and consider class weights, resampling or SMOTE.

Activation functions: ReLU (max of 0 and x) is the default in hidden layers; sigmoid squashes to 0 to 1 for binary outputs; softmax turns a vector into probabilities that sum to 1 for multi-class outputs; tanh squashes to -1 to 1.

CNNs use convolution filters to detect local patterns and share weights across the image, which suits vision tasks. ResNet adds skip connections so very deep networks can train without the vanishing gradient problem. Transfer learning reuses a pretrained network and fine-tunes it on a small dataset.

RNNs and LSTMs process sequences step by step and struggle with long dependencies because of vanishing gradients; LSTM gates help but transformers largely replaced them.

Tree models: decision trees split on features to reduce impurity; random forests average many trees trained on random subsets (bagging); XGBoost builds trees sequentially, each correcting the errors of the previous ones (boosting). Feature scaling matters for distance-based and gradient-based models but not for tree models.